

EXPERIMENT 17 — SUM OF INTEGERS FROM 1 TO 10

Aim

To write and execute a Prolog program to find the sum of integers from 1 to 10.

Objective

- To understand recursion in Prolog.
- To perform arithmetic operations.
- To calculate the sum of a sequence of integers.

Algorithm

1. Start.
2. Define the base case $\text{sum}(1,1)$.
3. For $N > 1$, calculate $N-1$.
4. Recursively calculate the sum up to $N-1$.
5. Add N to the previous sum.
6. Display the result.
7. Stop.

Flowchart

START

|

v

Set $N = 10$

|

v

Is $N = 1$?

/ \

Yes No

| |

Return 1 Calculate $N-1$

| |

| v

| Recursive Sum

```

|      |
|      v
|      S = N + S1
|      |
|      |
\      /
v      v
Display Sum
|
v
STOP

```

Prolog Program

sum(1, 1).

sum(N, S) :-

```

    N > 1,
    N1 is N - 1,
    sum(N1, S1),
    S is N + S1.

```

Query

?- sum(10, S).

Sample Output

S = 55.

```

SWI-Prolog console
File Settings Tools Debug Help
Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.2)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

3 ?- consult(['c:/Users/sadha/OneDrive/Desktop/exp-17.pl']).
true.

4 ?- sum(10, S).
S = 55 ▲

```

Result

Thus, the Prolog program to calculate the sum of integers from 1 to 10 was successfully executed.

EXPERIMENT 18 — NAME AND DOB DATABASE

Aim

To create a Prolog database containing names and dates of birth and retrieve information using queries.

Objective

- To understand facts in Prolog.
- To create a simple database.
- To retrieve information using variables.

Algorithm

1. Start.
2. Define person facts containing name and DOB.
3. Query the database using a name.
4. Retrieve the corresponding DOB.
5. Alternatively, query using DOB to find the name.
6. Stop.

Flowchart

START

|

v

Create Name-DOB Facts

|

v

Enter Query

|

v

Search Database

|

v

Matching Fact?

/ \

Yes No

| |

Display False

Answer

|

v

STOP

Prolog Program

person(john, '10-05-2000').

person(alice, '15-08-2001').

person(bob, '20-12-1999').

person(sara, '25-03-2002').

Queries

?- person(john, DOB).

Output

DOB = '10-05-2000'.

Another query:

?- person(Name, '15-08-2001').

Output

Name = alice.

```
SWI-Prolog console
File Settings Tools Debug Help
Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.2)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

7 ?- consult(['c:/Users/sadha/OneDrive/Desktop/exp-18.pl']).
true.

8 ?- person(john, DOB).
DOB = '10-05-2000'.

9 ?- person(Name, '15-08-2001').
Name = alice.

10 ?- ▲
```

Result

Thus, the Name-DOB database was successfully created and queried using Prolog.

EXPERIMENT 19 — STUDENT-TEACHER-SUBJECT CODE

Aim

To create a Prolog database representing students, teachers, subjects and subject codes.

Objective

- To represent relationships using Prolog facts.
- To retrieve teacher, student and subject information.
- To understand predicates with multiple arguments.

Algorithm

1. Start.
2. Define student facts.
3. Define teacher facts.
4. Define subject and subject-code facts.
5. Define teaching relationships.

6. Define student-subject relationships.
7. Execute queries.
8. Stop.

Flowchart

START

|

v

Define Students

|

v

Define Teachers

|

v

Define Subjects & Codes

|

v

Define Relationships

|

v

Enter Query

|

v

Display Matching Data

|

v

STOP

Prolog Program

student(rahul).

student(priya).

student(anu).

teacher(ravi).

teacher(suresh).

subject(ai, ai101).

subject(ml, ml102).

subject(dbms, db103).

teaches(ravi, ai).

teaches(suresh, ml).

studies(rahul, ai).

studies(priya, ml).

studies(anu, dbms).

Queries

?- studies(rahul, Subject).

Output:

Subject = ai.

?- teaches(Teacher, ml).

Output:

Teacher = suresh.

?- subject(ai, Code).

Output:

Code = ai101.

```
SWI-Prolog console
File Settings Tools Debug Help
Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.2)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

24 ?- consult(['c:/Users/sadha/OneDrive/Desktop/exp-19.pl']).
true.

25 ?- studies(rahul, Subject).
Subject = ai.

26 ?- teaches(Teacher, ml).
Teacher = suresh.

27 ?- subject(ai, Code).
Code = ai101.

28 ?-  ▲
```

Result

Thus, the Student-Teacher-Subject database was successfully implemented using Prolog.

EXPERIMENT 20 — PLANET DATABASE

Aim

To create a Prolog database containing information about planets.

Objective

- To represent planetary information as facts.
- To retrieve planet information using queries.
- To understand database searching in Prolog.

Algorithm

1. Start.
2. Define planets with their size and type.
3. Enter a query.
4. Search the database.
5. Display matching planets.
6. Stop.

Flowchart

START

|

v

Create Planet Database

|

v

Enter Query

|

v

Search Facts

|

v

Display Matching Planet

|

v

STOP

Prolog Program

planet(mercury, small, rocky).

planet(venus, medium, rocky).

planet(earth, medium, rocky).

planet(mars, small, rocky).

planet(jupiter, large, gaseous).

planet(saturn, large, gaseous).

planet(uranus, large, gaseous).

planet(neptune, large, gaseous).

Queries

?- planet(earth, Size, Type).

Output:

Size = medium,

Type = rocky.

?- planet(X, large, gaseous).

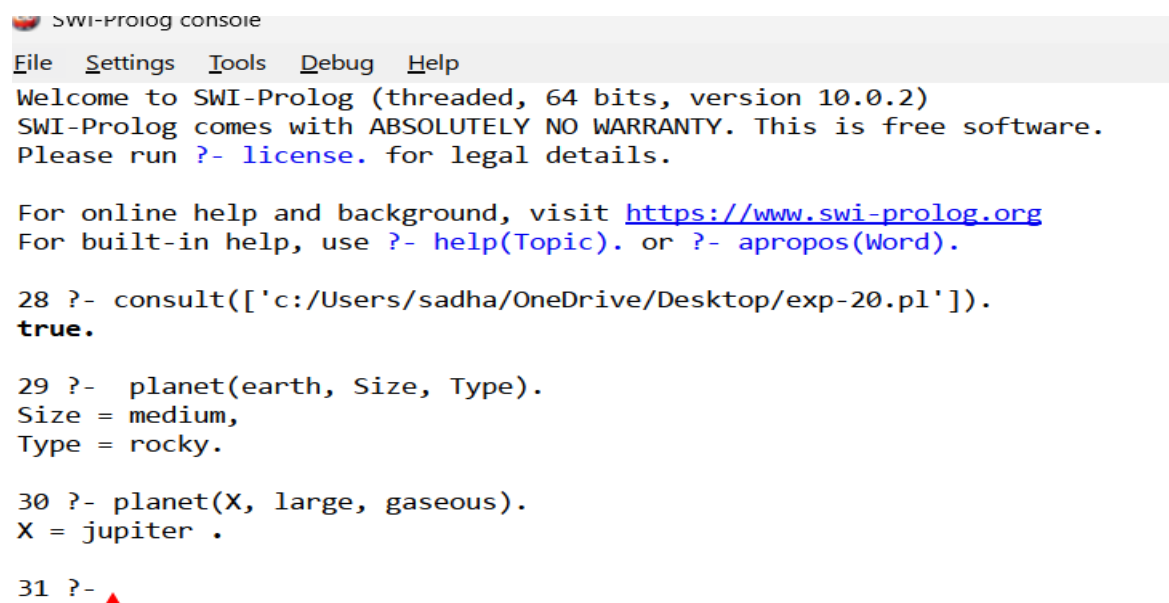
Output:

X = jupiter ;

X = saturn ;

X = uranus ;

X = neptune.



```
SWI-Prolog console
File Settings Tools Debug Help
Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.2)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

28 ?- consult(['c:/Users/sadha/OneDrive/Desktop/exp-20.pl']).
true.

29 ?- planet(earth, Size, Type).
Size = medium,
Type = rocky.

30 ?- planet(X, large, gaseous).
X = jupiter .

31 ?- ▲
```

Result

Thus, the Planet database was successfully implemented in Prolog.

EXPERIMENT 21 — TOWERS OF HANOI

Aim

To implement the Towers of Hanoi problem using recursion in Prolog.

Objective

- To understand recursion.
- To solve a classic AI problem.
- To generate the sequence of disk movements.

Algorithm

1. If there is one disk, move it directly from source to destination.
2. Otherwise, move N-1 disks from source to auxiliary.
3. Move the remaining disk from source to destination.
4. Move N-1 disks from auxiliary to destination.
5. Repeat until all disks are moved.

Flowchart

START

|

v

Input N

|

v

Is N = 1?

/ \

Yes No

| |

Move Disk Move N-1

| |

| Move Largest

| |

| Move N-1

\ /

v v

Display Moves

|

v

STOP

Prolog Program

hanoi(1, Source, Destination, _) :-

```
write('Move disk from '),  
write(Source),  
write(' to '),  
write(Destination),  
nl.
```

hanoi(N, Source, Destination, Auxiliary) :-

```
    N > 1,  
    N1 is N - 1,  
    hanoi(N1, Source, Auxiliary, Destination),  
    hanoi(1, Source, Destination, Auxiliary),  
    hanoi(N1, Auxiliary, Destination, Source).
```

Query

?- hanoi(3, a, c, b).

Sample Output

```
Move disk from a to c  
Move disk from a to b  
Move disk from c to b  
Move disk from a to c  
Move disk from b to a  
Move disk from b to c  
Move disk from a to c  
true.
```

```
SWI-Prolog console
File Settings Tools Debug Help
Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.2)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

31 ?- consult(['c:/Users/sadha/OneDrive/Desktop/exp-21.pl']).
true.

32 ?- hanoi(3, a, c, b).
Move disk from a to c
Move disk from a to b
Move disk from c to b
Move disk from a to c
Move disk from b to a
Move disk from b to c
Move disk from a to c
true ▲
```

Result

Thus, the Towers of Hanoi problem was successfully implemented using recursion in Prolog.

EXPERIMENT 22 — BIRD CAN FLY OR NOT

Aim

To write a Prolog program to determine whether a particular bird can fly or cannot fly.

Objective

- To represent bird information using facts.
- To use rules for determining flying ability.
- To perform logical queries.

Algorithm

1. Start.
2. Define different birds.
3. Define birds that can fly.
4. Define birds that cannot fly.
5. Create rules flies and does_not_fly.
6. Query a particular bird.
7. Display the result.
8. Stop.

Flowchart

START

|

v

Input Bird

|

v

Is Bird in Can-Fly List?

/

\

Yes

No

|

|

Can Fly

Check Cannot-Fly

|

v

Cannot Fly

|

v

STOP

Prolog Program

bird(sparrow).

bird(eagle).

bird(pigeon).

bird(ostrich).

bird(penguin).

can_fly(sparrow).

can_fly(eagle).

can_fly(pigeon).

cannot_fly(ostrich).

```
cannot_fly(penguin).
```

```
flies(Bird) :-
```

```
    bird(Bird),  
    can_fly(Bird).
```

```
does_not_fly(Bird) :-
```

```
    bird(Bird),  
    cannot_fly(Bird).
```

Queries

```
?- flies(eagle).
```

Output:

```
true.
```

```
?- does_not_fly(penguin).
```

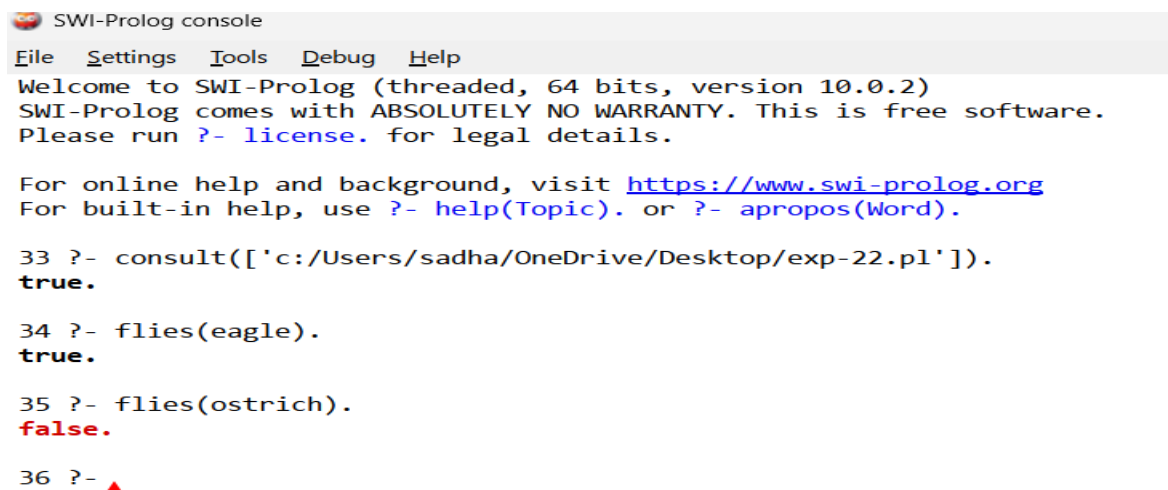
Output:

```
true.
```

```
?- flies(ostrich).
```

Output:

```
false.
```



```
SWI-Prolog console  
File Settings Tools Debug Help  
Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.2)  
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.  
Please run ?- license. for legal details.  
  
For online help and background, visit https://www.swi-prolog.org  
For built-in help, use ?- help(Topic). or ?- apropos(Word).  
  
33 ?- consult(['c:/Users/sadha/OneDrive/Desktop/exp-22.pl']).  
true.  
  
34 ?- flies(eagle).  
true.  
  
35 ?- flies(ostrich).  
false.  
  
36 ?- ▲
```

Result

Thus, the Prolog program successfully determines whether a particular bird can fly or not.

EXPERIMENT 23 — FAMILY TREE

Aim

To implement a family tree using facts and rules in Prolog.

Objective

- To represent family relationships.
- To implement father, mother and grandparent relationships.
- To retrieve relationships using queries.

Algorithm

1. Start.
2. Define male and female members.
3. Define parent relationships.
4. Define rules for father and mother.
5. Define the grandparent rule.
6. Execute queries.
7. Display results.
8. Stop.

Flowchart

START

|

v

Define Family Members

|

v

Define Parent Relations

|

v

Apply Family Rules

|

v

Enter Query

|

v

Display Relationship

|

v

STOP

Prolog Program

male(john).

male(peter).

male(david).

female(mary).

female(anna).

female(lisa).

parent(john, peter).

parent(mary, peter).

parent(peter, david).

parent(anna, david).

parent(peter, lisa).

parent(anna, lisa).

father(X, Y) :-

male(X),

parent(X, Y).

mother(X, Y) :-

female(X),

```
parent(X, Y).
```

```
grandparent(X, Y) :-
```

```
    parent(X, Z),
```

```
    parent(Z, Y).
```

Queries

```
?- father(john, peter).
```

Output:

```
true.
```

```
?- mother(mary, peter).
```

Output:

```
true.
```

```
?- grandparent(john, david).
```

Output:

```
true.
```



```
SWI-Prolog console
File Settings Tools Debug Help
Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.2)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

36 ?- consult(['c:/Users/sadha/OneDrive/Desktop/exp-23.pl']).
true.

37 ?- father(john, peter).
true.

38 ?- grandparent(X, david).
X = john .

39 ?- mother(mary, peter).
true.
```

Result

Thus, the family tree was successfully implemented using Prolog.

EXPERIMENT 24 — DIET SUGGESTION BASED ON DISEASE

Aim

To develop a Prolog-based diet suggestion system according to a disease.

Objective

- To create disease-diet knowledge.
- To use rules for providing suggestions.
- To retrieve appropriate diet information.

Algorithm

1. Start.
2. Define diseases and corresponding diets.
3. Define suggest_diet rule.
4. Enter disease as a query.
5. Retrieve the corresponding diet.
6. Display the recommendation.
7. Stop.

Flowchart

START

|

v

Enter Disease

|

v

Search Disease Database

|

v

Disease Found?

/ \

Yes No

| |

Display False

Diet

|

v

STOP

Prolog Program

diet(diabetes, 'Low sugar and high fiber diet').

diet(hypertension, 'Low salt and low fat diet').

diet(obesity, 'Low calorie and high protein diet').

diet(anemia, 'Iron rich foods and green vegetables').

diet(cold, 'Warm fluids, fruits and vegetables').

suggest_diet(Disease, Diet) :-

diet(Disease, Diet).

Queries

?- suggest_diet(diabetes, Diet).

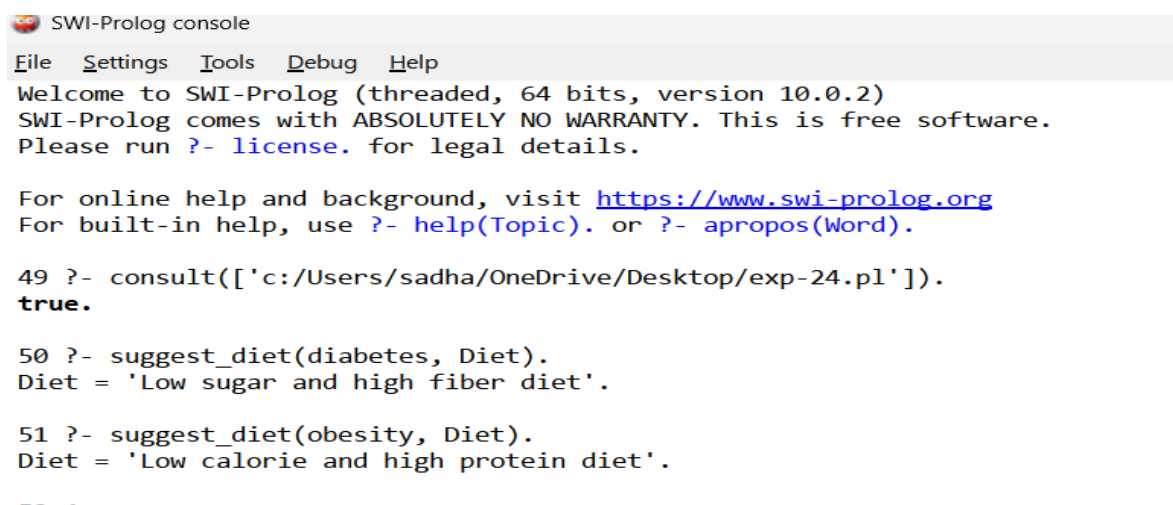
Output:

Diet = 'Low sugar and high fiber diet'.

?- suggest_diet(obesity, Diet).

Output:

Diet = 'Low calorie and high protein diet'.



```
SWI-Prolog console
File Settings Tools Debug Help
Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.2)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

49 ?- consult(['c:/Users/sadha/OneDrive/Desktop/exp-24.pl']).
true.

50 ?- suggest_diet(diabetes, Diet).
Diet = 'Low sugar and high fiber diet'.

51 ?- suggest_diet(obesity, Diet).
Diet = 'Low calorie and high protein diet'.

-- -
```

Result

Thus, the disease-based diet suggestion system was successfully implemented in Prolog.

EXPERIMENT 25 — MONKEY BANANA PROBLEM

Aim

To implement the Monkey Banana Problem using state representation in Prolog.

Objective

- To understand state-space representation.
- To represent monkey, box and banana states.
- To implement actions such as climbing and obtaining the banana.

Algorithm

1. Start.
2. Represent the initial state.
3. Represent monkey movement.
4. Represent box movement.
5. Represent climbing onto the box.
6. Represent getting the banana.
7. Display the sequence of actions.
8. Stop.

Flowchart

START

|

v

Monkey at Position

|

v

Move Box

|

v

Monkey Reaches Box

|

v

Climb Box

|

v

Get Banana

|

v

STOP

Prolog Program

```
move(state(middle, onfloor, middle, hasnot),
      state(middle, onbox, middle, hasnot)) :-
  write('Monkey climbs the box'), nl.
```

```
move(state(Pos, onfloor, Pos, Has),
      state(NewPos, onfloor, NewPos, Has)) :-
  Pos \= NewPos,
  write('Monkey pushes box from '),
  write(Pos),
  write(' to '),
  write(NewPos), nl.
```

```
move(state(Pos, onbox, Pos, hasnot),
      state(Pos, onbox, Pos, has)) :-
  write('Monkey gets banana'), nl.
```

Queries

Climb the box

```
?- move(state(middle,onfloor,middle,hasnot),
        state(middle,onbox,middle,hasnot)).
```

Output:

Monkey climbs the box

true.

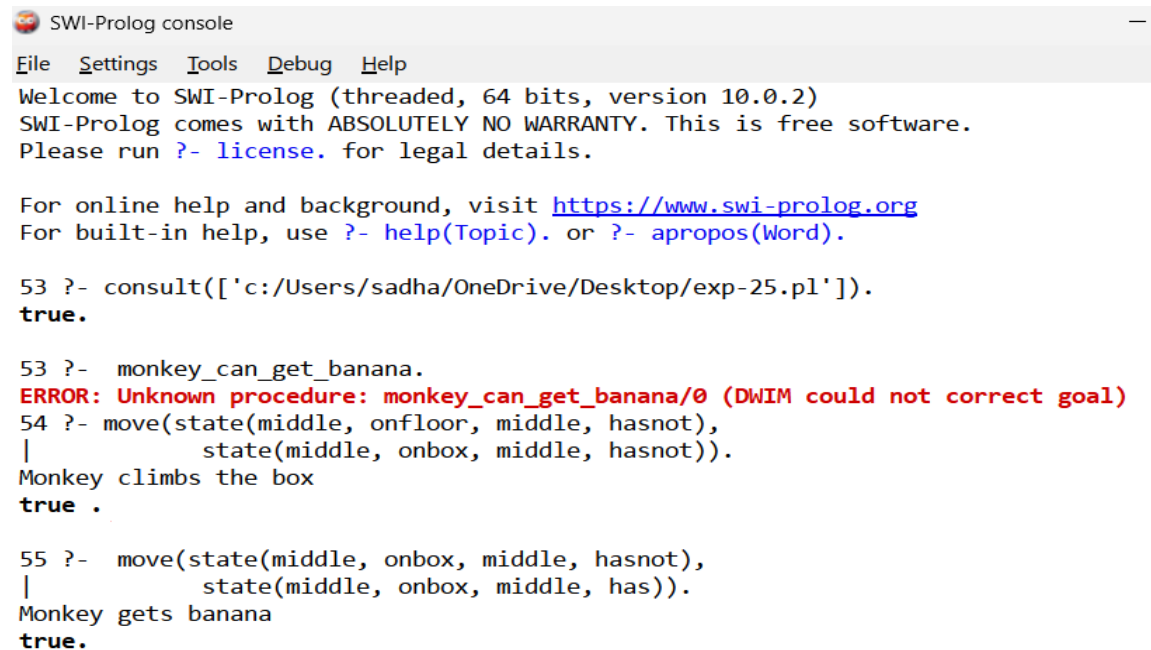
Get banana

```
?- move(state(middle,onbox,middle,hasnot),  
        state(middle,onbox,middle,has)).
```

Output:

Monkey gets banana

true.



```
SWI-Prolog console  
File Settings Tools Debug Help  
Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.2)  
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.  
Please run ?- license. for legal details.  
  
For online help and background, visit https://www.swi-prolog.org  
For built-in help, use ?- help(Topic). or ?- apropos(Word).  
  
53 ?- consult(['c:/Users/sadha/OneDrive/Desktop/exp-25.pl']).  
true.  
  
53 ?- monkey_can_get_banana.  
ERROR: Unknown procedure: monkey_can_get_banana/0 (DWIM could not correct goal)  
54 ?- move(state(middle, onfloor, middle, hasnot),  
|         state(middle, onbox, middle, hasnot)).  
Monkey climbs the box  
true .  
  
55 ?- move(state(middle, onbox, middle, hasnot),  
|         state(middle, onbox, middle, has)).  
Monkey gets banana  
true.
```

Result

Thus, the Monkey Banana Problem was successfully implemented using state representation in Prolog.

EXPERIMENT 26 — FRUIT AND COLOR USING BACKTRACKING

Aim

To implement a Prolog program to identify fruits and their colors using backtracking.

Objective

- To understand backtracking in Prolog.
- To retrieve multiple solutions.
- To represent fruit-color relationships.

Algorithm

1. Start.
2. Define fruits and their colors.

3. Query the database with a variable.
4. Prolog finds the first matching fact.
5. Press ; to request another solution.
6. Continue until all solutions are obtained.
7. Stop.

Flowchart

START

|

v

Define Fruit-Color Facts

|

v

Enter Query

|

v

Find Matching Fact

|

v

More Solutions?

/ \

Yes No

|

|

Press ; STOP

|

v

Next Solution

Prolog Program

fruit(apple, red).

fruit(banana, yellow).

fruit(orange, orange).

fruit(grapes, green).

fruit(mango, yellow).

fruit(watermelon, green).

Query

?- fruit(Fruit, Color).

Sample Output

Fruit = apple,

Color = red ;

Fruit = banana,

Color = yellow ;

Fruit = orange,

Color = orange ;

Fruit = grapes,

Color = green ;

Fruit = mango,

Color = yellow ;

Fruit = watermelon,

Color = green.

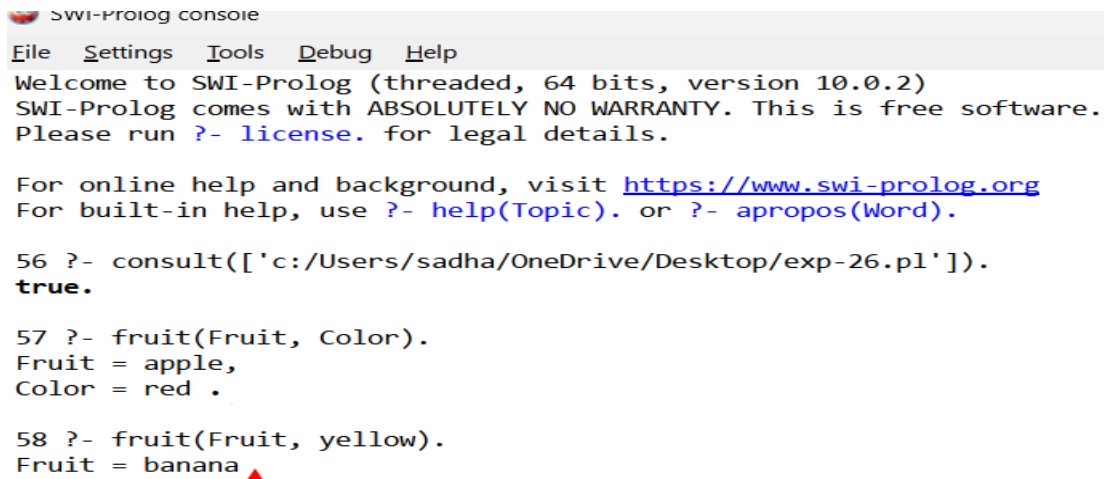
Query for yellow fruits

?- fruit(Fruit, yellow).

Output:

Fruit = banana ;

Fruit = mango.

A screenshot of the SWI-Prolog console window. The title bar says "SWI-Prolog console". The menu bar includes "File", "Settings", "Tools", "Debug", and "Help". The main text area contains the following: "Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.2)", "SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software. Please run ?- license. for legal details.", "For online help and background, visit <https://www.swi-prolog.org>", "For built-in help, use ?- help(Topic). or ?- apropos(Word).", "56 ?- consult(['c:/Users/sadha/OneDrive/Desktop/exp-26.pl']).", "true.", "57 ?- fruit(Fruit, Color).", "Fruit = apple,", "Color = red .", "58 ?- fruit(Fruit, yellow).", "Fruit = banana ▲".

```
SWI-Prolog console
File Settings Tools Debug Help
Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.2)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

56 ?- consult(['c:/Users/sadha/OneDrive/Desktop/exp-26.pl']).
true.

57 ?- fruit(Fruit, Color).
Fruit = apple,
Color = red .

58 ?- fruit(Fruit, yellow).
Fruit = banana ▲
```

Result

Thus, the fruit-color relationship was successfully implemented using Prolog backtracking.

EXPERIMENT 27 — BEST FIRST SEARCH

Aim

To implement the Best First Search algorithm using Prolog.

Objective

- To understand heuristic search.
- To represent a graph using edges.
- To select nodes based on heuristic values.
- To find a path from start to goal.

Algorithm

1. Start.
2. Define the graph.
3. Assign heuristic values to nodes.
4. Place the start node in the OPEN list.
5. Select the node with the lowest heuristic value.
6. Expand the selected node.
7. Add its successors to the OPEN list.

8. Repeat until the goal is reached.
9. Display the path.
10. Stop.

Flowchart

START

|

v

Define Graph

|

v

Define Heuristic Values

|

v

Put Start in OPEN

|

v

Select Lowest Heuristic

|

v

Is it Goal?

/ \

Yes No

| |

Display Expand Node

Path |

| v

| Add Successors

| |

\ /

v v

Repeat

|

v

STOP

Prolog Program

edge(a, b).

edge(a, c).

edge(b, d).

edge(c, d).

edge(c, e).

edge(d, e).

heuristic(a, 6).

heuristic(b, 4).

heuristic(c, 3).

heuristic(d, 1).

heuristic(e, 0).

best_first(Start, Goal, Path) :-

search([[Start]], Goal, Path).

search([[Goal|Rest]|_], Goal, Path) :-

reverse([Goal|Rest], Path).

search([[Current|Rest]|Paths], Goal, Path) :-

findall(

[Next,Current|Rest],

(edge(Current, Next),

\+ member(Next, [Current|Rest])),

NewPaths

```

),
append(Paths, NewPaths, AllPaths),
sort_paths(AllPaths, SortedPaths),
search(SortedPaths, Goal, Path).

```

sort_paths(Paths, Sorted) :-

```

    map_list_to_pairs(path_cost, Paths, Pairs),
    keysort(Pairs, SortedPairs),
    pairs_values(SortedPairs, Sorted).

```

path_cost([Node|_], H) :-

```

    heuristic(Node, H).

```

Query

```
?- best_first(a, e, Path).
```

Sample Output

Path = [a,c,e].



```

SWI-Prolog console
File Settings Tools Debug Help
Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.2)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

59 ?- consult(['c:/Users/sadha/OneDrive/Desktop/exp-27.pl']).
true.

60 ?- best_first(a, e, Path).
Path = [a, c, e] ▲

```

Result

Thus, the Best First Search algorithm was successfully implemented using Prolog.

EXPERIMENT 28 — MEDICAL DIAGNOSIS

Aim

To implement a simple medical diagnosis system using Prolog.

Objective

- To represent symptoms as facts.
- To use rules for diagnosis.
- To infer possible diseases from symptoms.

Algorithm

1. Start.
2. Define symptoms.
3. Define disease rules.
4. Match symptoms against rules.
5. Infer the disease.
6. Display the diagnosis.
7. Stop.

Flowchart

START

|

v

Enter Symptoms

|

v

Compare with Rules

|

v

Symptoms Match?

/ \

Yes No

| |

Diagnosis No Diagnosis

|

v

STOP

Prolog Program

symptom(fever).

symptom(cough).

symptom(headache).

symptom(body_pain).

diagnosis(flu) :-

 symptom(fever),

 symptom(cough),

 symptom(body_pain).

diagnosis(cold) :-

 symptom(cough),

 symptom(headache).

diagnosis(dengue) :-

 symptom(fever),

 symptom(headache),

 symptom(body_pain).

Queries

?- diagnosis(flu).

Output:

true.

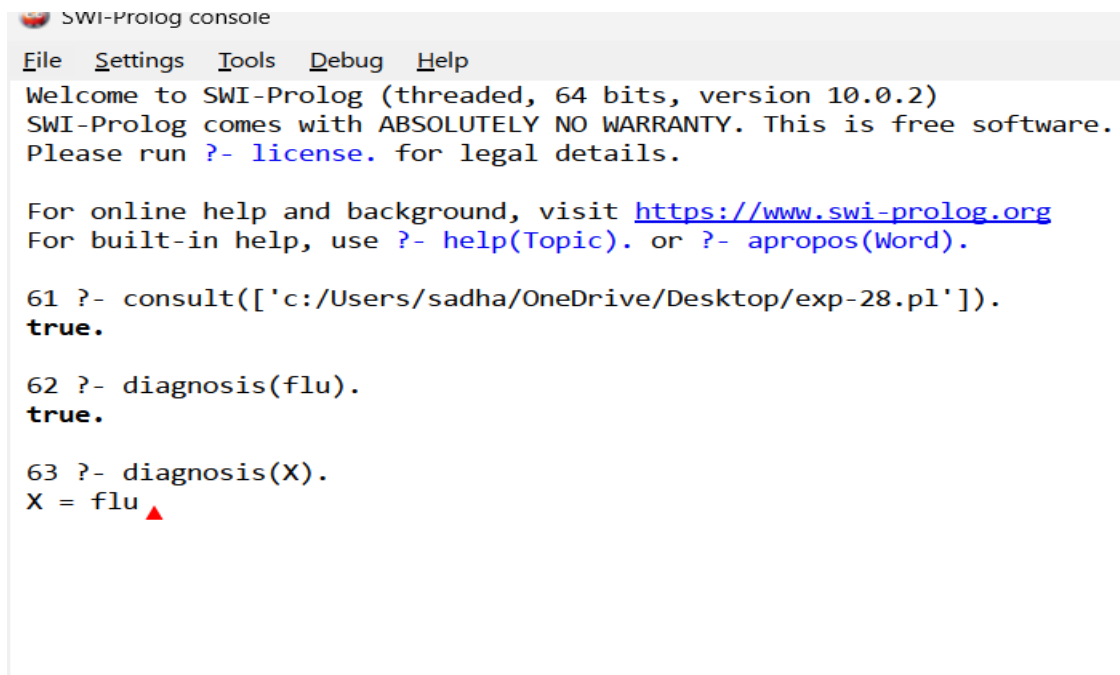
?- diagnosis(X).

Possible output:

X = flu ;

X = cold ;

X = dengue.



```
SWI-Prolog console
File Settings Tools Debug Help
Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.2)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

61 ?- consult(['c:/Users/sadha/OneDrive/Desktop/exp-28.pl']).
true.

62 ?- diagnosis(flu).
true.

63 ?- diagnosis(X).
X = flu ▲
```

Result

Thus, a simple medical diagnosis system was successfully implemented using Prolog.

EXPERIMENT 29 — FORWARD CHAINING

Aim

To implement forward chaining using Prolog.

Objective

- To understand data-driven reasoning.
- To start reasoning from known facts.
- To derive new facts using rules.

Algorithm

1. Start.
2. Define initial facts.
3. Define rules.
4. Select a known fact.
5. Apply a matching rule.
6. Generate a new fact.

7. Continue until the required conclusion is obtained.

8. Stop.

Flowchart

START

|

v

Initial Facts

|

v

Check Rules

|

v

Rule Condition Satisfied?

/ \

Yes No

| |

Generate New Check Next

Fact Rule

|

v

Goal Reached?

/ \

Yes No

| |

STOP Repeat

Prolog Program

fact(sunny).

fact(warm).

rule(sunny, weather_good).

```
rule(weather_good, go_outside).
```

forward :-

```
    fact(sunny),  
    rule(sunny, weather_good),  
    write('Derived: weather_good'), nl,  
    rule(weather_good, go_outside),  
    write('Derived: go_outside'), nl.
```

Query

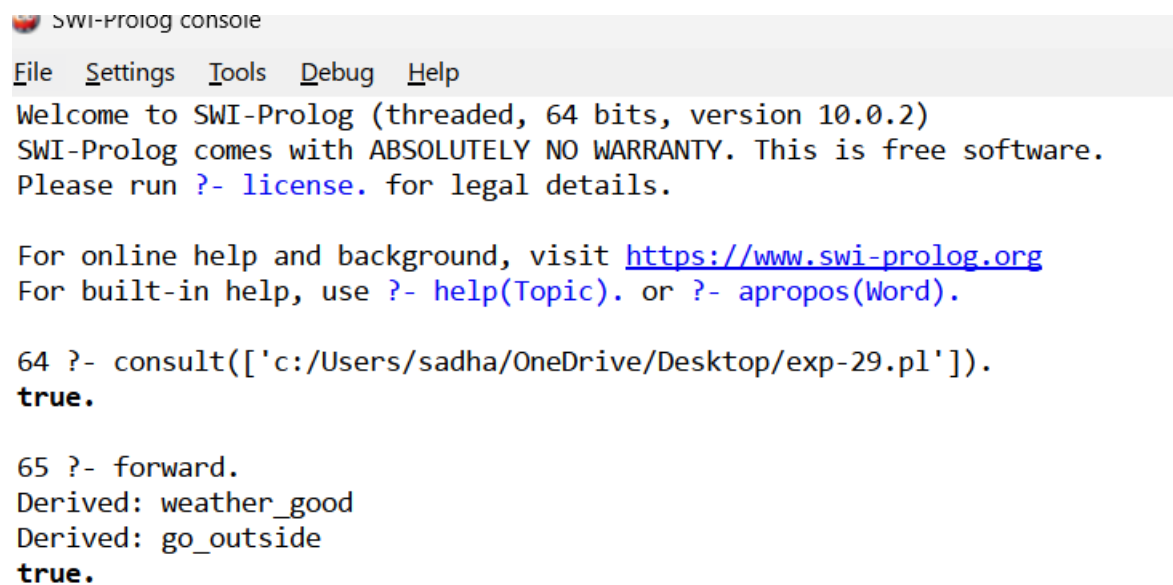
?- forward.

Sample Output

Derived: weather_good

Derived: go_outside

true.



```
SWI-Prolog console  
File Settings Tools Debug Help  
Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.2)  
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.  
Please run ?- license. for legal details.  
  
For online help and background, visit https://www.swi-prolog.org  
For built-in help, use ?- help(Topic). or ?- apropos(Word).  
  
64 ?- consult(['c:/Users/sadha/OneDrive/Desktop/exp-29.pl']).  
true.  
  
65 ?- forward.  
Derived: weather_good  
Derived: go_outside  
true.
```

Result

Thus, forward chaining was successfully implemented using Prolog.

EXPERIMENT 30 — BACKWARD CHAINING

Aim

To implement backward chaining using Prolog.

Objective

- To understand goal-driven reasoning.
- To start from a goal.
- To find facts that support the goal.

Algorithm

1. Start.
2. Define facts.
3. Define rules.
4. Specify a goal.
5. Check whether a rule can prove the goal.
6. Check the conditions of that rule.
7. Continue backward until facts are found.
8. Display the result.
9. Stop.

Flowchart

START

|

v

Set Goal

|

v

Find Rule for Goal

|

v

Check Rule Conditions

|

v

Are Conditions Facts?

/ \

Yes No

| |

Prove Goal Find Rules

| |

\ /

v v

Goal Proven

|

v

STOP

Prolog Program

fact(rainy).

fact(cold).

wear(jacket) :-

 fact(cold).

stay_home :-

 fact(rainy).

go_outside :-

 fact(rainy),

 fact(cold).

Queries

?- wear(jacket).

Output:

true.

?- stay_home.

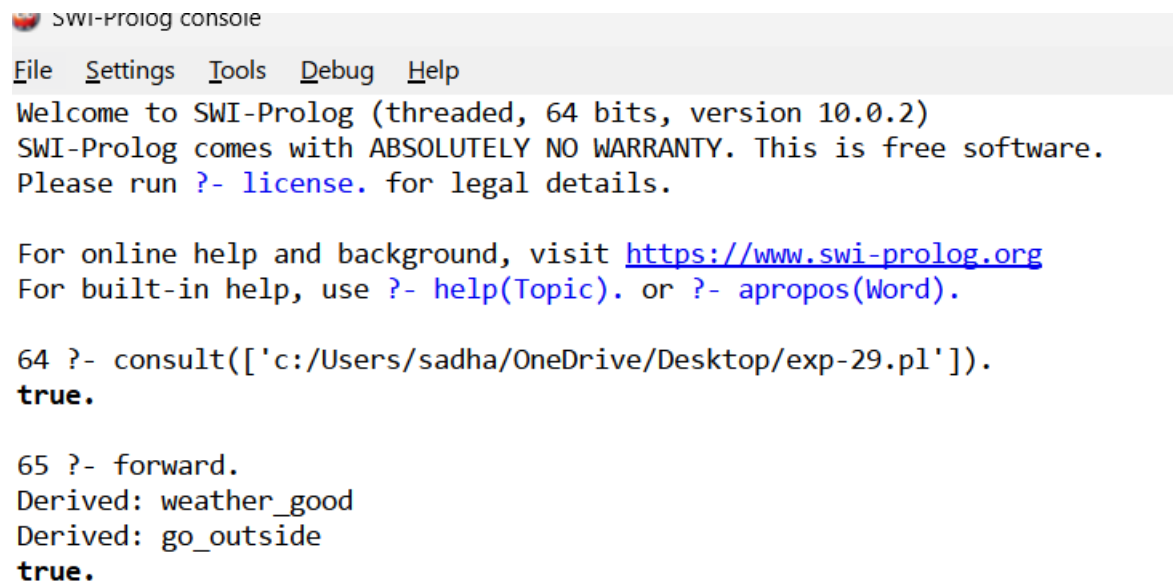
Output:

true.

?- go_outside.

Output:

true.



```
SWI-Prolog console
File Settings Tools Debug Help
Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.2)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

64 ?- consult(['c:/Users/sadha/OneDrive/Desktop/exp-29.pl']).
true.

65 ?- forward.
Derived: weather_good
Derived: go_outside
true.
```

Result

Thus, backward chaining was successfully implemented using Prolog.